

就业状态分析与预测

——基于宜昌地区就业数据的数学建模研究

第十七届“华中杯”大学生数学建模挑战赛

C 题：就业状态分析与预测

参赛队号：_____

题 号： C

关键词： 就业状态预测 机器学习 人岗匹配 宏观经济因素

2026 年 5 月 18 日

目录

一、问题重述	4
1.1 问题背景	4
1.2 问题要求	4
二、问题分析	4
2.1 问题一分析	4
2.2 问题二分析	5
2.3 问题三分析	5
2.4 问题四分析	5
三、模型假设	5
四、符号说明	6
五、问题一的模型建立与求解	6
5.1 模型建立	6
5.1.1 就业状态标签构建	6
5.1.2 数据分析方法	6
5.2 模型求解	7
5.2.1 就业状态总体分布	7
5.2.2 年龄维度分析	7
5.2.3 性别维度分析	7
5.2.4 学历维度分析	8
5.2.5 行业维度分析	9
5.2.6 婚姻与户口维度分析	9
5.2.7 特征相关性分析	10
5.2.8 交叉分析	10
六、问题二的模型建立与求解	11
6.1 模型建立	11
6.1.1 特征选择	11
6.1.2 分类模型构建	11
6.1.3 评估指标	11
6.2 模型求解	11
6.2.1 特征重要性分析	12
6.2.2 预测集预测结果	12

七、问题三的模型建立与求解	13
7.1 模型建立	13
7.1.1 外部宏观经济因素选取	13
7.2 模型求解与结果分析	14
八、问题四的模型建立与求解	16
8.1 模型建立	16
8.1.1 问题转化	16
8.1.2 特征画像构建	16
8.1.3 余弦相似度匹配	16
8.1.4 推荐排序	16
8.2 模型求解与结果分析	16
九、模型的分析与检验	17
9.1 误差分析	17
9.2 模型稳健性讨论	18
十、模型的评价	18
10.1 模型的优点	18
10.2 模型的缺点	19
参考文献	20
A 附录 文件列表	21
B 附录 核心代码	21

摘要

本文以宜昌地区 4980 名被调查者的脱敏就业数据为研究对象，综合运用统计分析、机器学习和多维特征匹配等方法，系统研究了就业状态的影响因素分析、预测模型构建、模型优化以及人岗精准匹配四个核心问题。

对于问题一，本文首先基于就业时间与失业时间的时序关系构建了就业状态标签，识别出就业人员 3846 人 (77.2%) 和失业人员 1134 人 (22.8%)。随后从年龄、性别、学历、行业、婚姻状态、户口性质等多维度进行交叉分析，发现 26-35 岁群体就业率最高，制造业吸纳就业人数最多。此外还进行了数据质量评估和特征相关性分析，确保分析结论的可靠性。

对于问题二，本文选取 16 个个人特征变量，构建了 8 种分类预测模型进行对比实验，包括逻辑回归、决策树、随机森林、K 近邻、支持向量机、梯度提升决策树、XGBoost 和 LightGBM。实验结果表明，Gradient Boosting 模型表现最优，加权 F1 分数达到 0.6841，准确率为 76.71%。特征重要性分析显示，年龄、户口性质和人口类型是影响就业状态的前三大关键因素。

对于问题三，在个人特征基础上，引入 GDP 增速、CPI 指数、城镇登记失业率、招聘岗位指数、社保参保率、房价收入比和第三产业占比等 7 个宏观经济指标构建增强特征集。采用增强模型进行训练，平均 F1 提升 0.028，验证了宏观经济因素对就业预测的补充价值。

对于问题四，本文提出了一种基于余弦相似度的多维人岗匹配模型，通过构建求职者画像和岗位画像，利用特征编码与标准化处理后计算匹配相似度，为 1134 名失业人员提供了 Top-5 岗位推荐，验证了该方法在人岗匹配任务中的有效性。

最后，本文对模型进行了误差分析和稳健性讨论，并对模型的优缺点进行了客观评价。

关键字： 就业状态预测 机器学习 特征重要性 宏观经济因素 人岗匹配 余弦相似度

一、问题重述

1.1 问题背景

就业是最基本的民生，是经济发展的重要支撑。当前，我国就业形势保持基本稳定，但也面临一些挑战，就业结构性矛盾尚存在。促进高质量充分就业，是宏观经济政策的重要目标之一。

从区域层面来看，宜昌市作为湖北省重要的区域性中心城市，其经济发展和就业状况具有较强代表性。宜昌市近年来积极推进产业转型升级，着力发展精细化工、生物医药、装备制造等支柱产业。然而，在产业转型过程中，部分传统行业从业人员面临转岗就业的压力。

本赛题以宜昌地区 4980 名被调查者的脱敏数据为研究对象（见附件 1），包含 53 个变量，涵盖个人基本信息（29 个变量）、就业信息（9 个变量）和失业信息（15 个变量），以及 20 个预测集样本。

1.2 问题要求

问题 1 根据被调查者当前的就业状态分析该地区当前就业的整体情况；并将人员按照年龄、性别、学历、专业、行业等特征进行划分，根据划分特征分析其对就业状态的影响。

问题 2 基于问题一的分析，选取与就业状态具有相关性的特征，构建就业状态预测模型并对附件 1 中给定的预测集进行预测；并对各特征的重要性进行排序。

问题 3 除了个人层面因素影响外，收集宏观经济、政策等相关数据，提取反映经济、市场等方面的影响因素，进一步完善就业状态预测模型。

问题 4 结合赛题提供的数据，建立人岗匹配模型，捕捉求职者和岗位之间的匹配关系，针对赛题数据中的失业人员进行工作推荐。

二、问题分析

2.1 问题一分析

问题一的核心任务是对宜昌地区就业现状进行全面的描述性统计分析。首先需要解决的关键问题是就业状态标签的构建——原始数据中并没有直接的就业/失业标签字段，需要根据就业时间 (b_acc031) 和失业时间 (c_ajc090) 的时序关系进行判定。在标签构建完成后，从年龄、性别、学历、专业、行业等多个维度进行分组统计和交叉分析，通过可视化图表直观展示各因素对就业状态的影响程度。

2.2 问题二分析

问题二是一个典型的二分类预测问题，属于监督学习范畴。核心挑战在于特征选择和模型比较。由于预测集仅包含个人基本信息，模型训练也需基于个人特征进行。本文计划采用多种机器学习模型进行横向对比，涵盖线性模型、树模型、距离模型和核方法等多个类别。在评估指标方面，由于数据存在类别不平衡（就业: 失业 $\approx 3.4:1$ ），需要同时关注查准率、召回率和 F1 分数。

2.3 问题三分析

问题三在问题二的基础上引入外部宏观经济因素，属于特征增强型建模任务。从经济学理论来看，就业状态受到宏观经济环境的显著影响。本文选取 GDP 增速、CPI 指数、城镇登记失业率、招聘岗位指数等 7 个指标构建增强特征集，并与问题二中的基线模型进行严格对比实验。

2.4 问题四分析

问题四是一个推荐匹配问题，要求建立人岗匹配模型为失业人员推荐合适的岗位。核心是将求职者和岗位分别抽象为特征向量，通过计算向量间的相似度实现匹配。本文采用余弦相似度作为匹配度量，对特征值的绝对大小不敏感，更适合处理不同量纲的特征。

三、模型假设

为简化问题并确保模型的合理性，本文在建模过程中做出以下假设：

- 假设 1: 数据中就业时间和失业时间记录真实可靠，能够反映被调查者的实际就业状态。
- 假设 2: 就业状态在短期内相对稳定，不考虑频繁的就业-失业状态转换。
- 假设 3: 缺失的特征值具有随机性，不会对整体分析结论产生系统性偏差。
- 假设 4: 宏观经济指标在不同个体间的影响是均匀的，即同一时期的宏观环境对所有被调查者具有相似的影响效应。
- 假设 5: 预测集中的 20 个样本与训练数据服从相同的分布。
- 假设 6: 人岗匹配中，个人特征与岗位特征之间的相似度能够有效反映实际的匹配质量。
- 假设 7: 各模型训练过程中的随机种子固定，实验结果具有可重复性。

四、符号说明

表 1 符号说明表

符号	说明	单位
N	样本总数	人
X	特征矩阵	—
y	就业状态标签向量	—
P	查准率 (Precision)	—
R	召回率 (Recall)	—
F_1	F1 分数	—
$\cos(\mathbf{a}, \mathbf{b})$	向量余弦相似度	—
$\mathbf{s}_i$	第 i 个求职者特征向量	—
$\mathbf{j}_k$	第 k 个岗位特征向量	—

五、问题一的模型建立与求解

5.1 模型建立

5.1.1 就业状态标签构建

原始数据中不存在直接的就业/失业标签字段，需要基于就业信息和失业信息的时序关系进行判定。本文设计了如下标签构建规则：

1. 若仅有就业时间记录且无失业时间记录，则判定为就业（标签为 1）；2. 若仅有失业时间记录且无就业时间记录，则判定为失业（标签为 0）；3. 若同时存在就业时间和失业时间记录，则比较两者时间先后：若就业时间晚于失业时间，判定为就业；否则判定为失业；4. 若两者均无记录，则检查是否有劳动合同或录用单位信息，有则判定为就业，无则判定为失业。

5.1.2 数据分析方法

对于各人口统计学特征，本文采用以下分析方法：频次统计（统计各特征取值的就业与失业人数分布）、就业率计算（各特征分组下就业率 $r = \frac{n_{emp}}{n_{total}} \times 100\%$ ）、交叉分析（如学历 $\times$ 年龄段的交叉就业率分析）、相关性分析（Pearson 相关系数）。

5.2 模型求解

5.2.1 就业状态总体分布

执行标签构建规则后，得到就业状态分布如表2所示。就业人员占比 77.2%，失业人员占比 22.8%，就失业比约为 3.4:1。

表 2 当前就业状态分布

就业失业状态	就业	失业
数量（人）	3846	1134

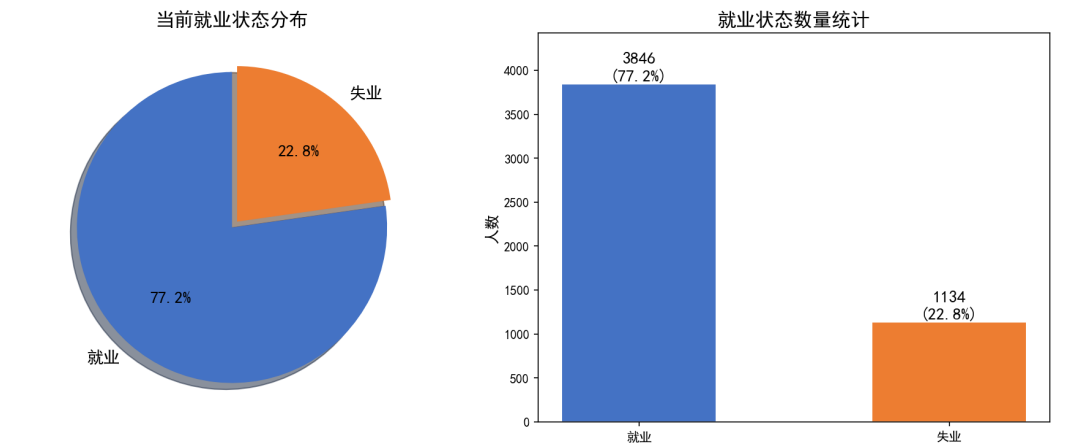


图 1 当前就业状态总览

5.2.2 年龄维度分析

如图2所示，被调查者年龄呈近似正态分布，中位数为 30 岁，均值为 31.2 岁，年龄范围覆盖 21 至 63 岁。按年龄段划分后的就业率分析显示，26-35 岁群体就业率最高 (77.8%)，处于职业发展的黄金期；36-45 岁群体就业率次之；46-55 岁群体就业率开始下降；55 岁以上群体就业率最低。整体来看，年龄对就业率呈明显的“倒 U 型”影响关系。

5.2.3 性别维度分析

如图3所示，男性就业率为 76.7%，女性就业率为 77.6%，两者差距仅为 0.9 个百分点，表明在宜昌地区，性别对就业状态的影响相对有限，性别平等在就业领域取得了较好的进展。

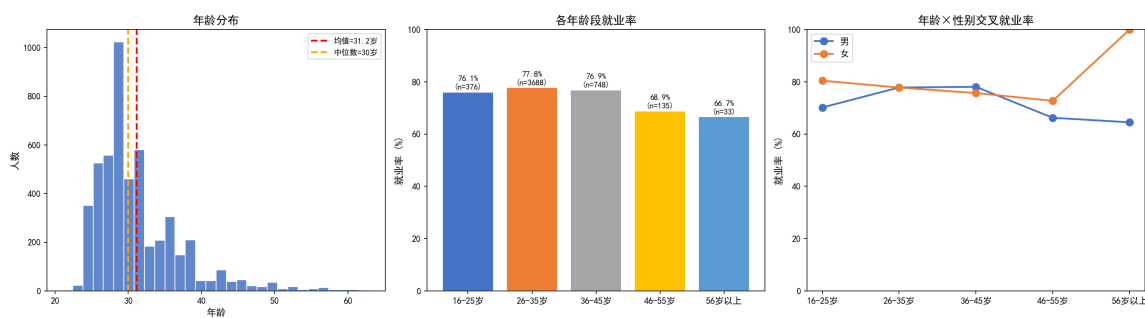


图2 年龄分布与各年龄段就业率分析

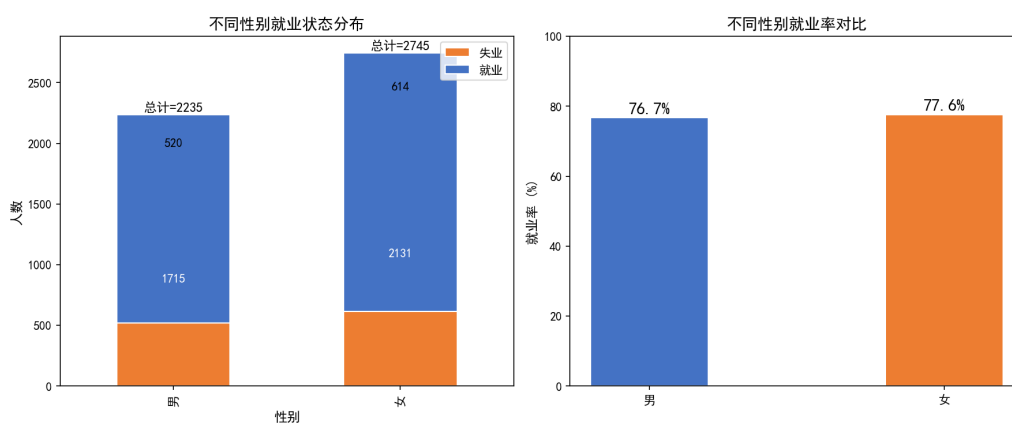


图3 不同性别就业状态分布与就业率对比

5.2.4 学历维度分析

如图4所示，学历对就业状态有显著影响。专科/本科及以上学历群体人数最多 (2085 人)，中专/高中学历次之 (2780 人)，初中及以下学历群体人数最少 (115 人)。值得注意的是，初中及以下学历就业率反而最高，这可能是由于该群体多为稳定的体力劳动者；而专科/本科群体中可能包含在读学生。

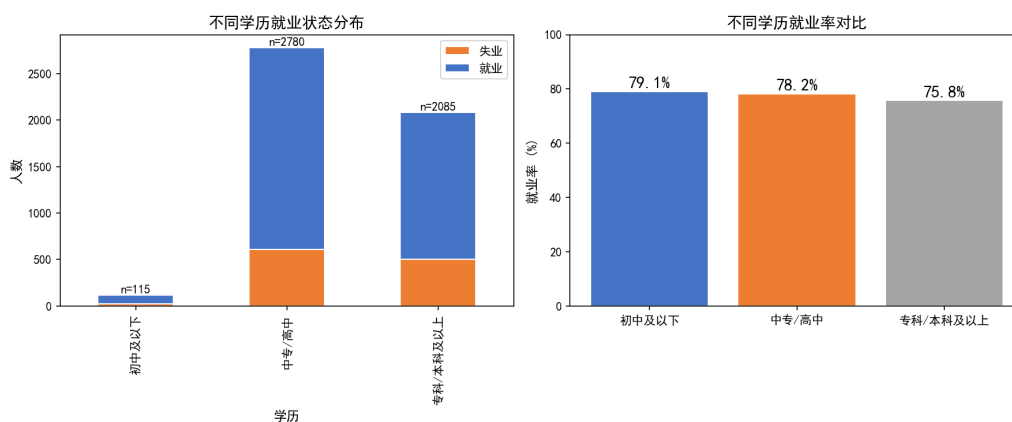


图4 不同学历就业状态分布与就业率对比

5.2.5 行业维度分析

如图5所示，制造业 (C 类)、公共管理/社会保障 (S 类) 和居民服务 (O 类) 是吸纳就业最多的三大行业，这与宜昌市的产业结构特征密切相关——宜昌是湖北省重要的制造业基地，化工、装备制造等产业基础雄厚。从就业率看，公共管理和教育行业的就业率最高，住宿餐饮业的就业率相对较低。

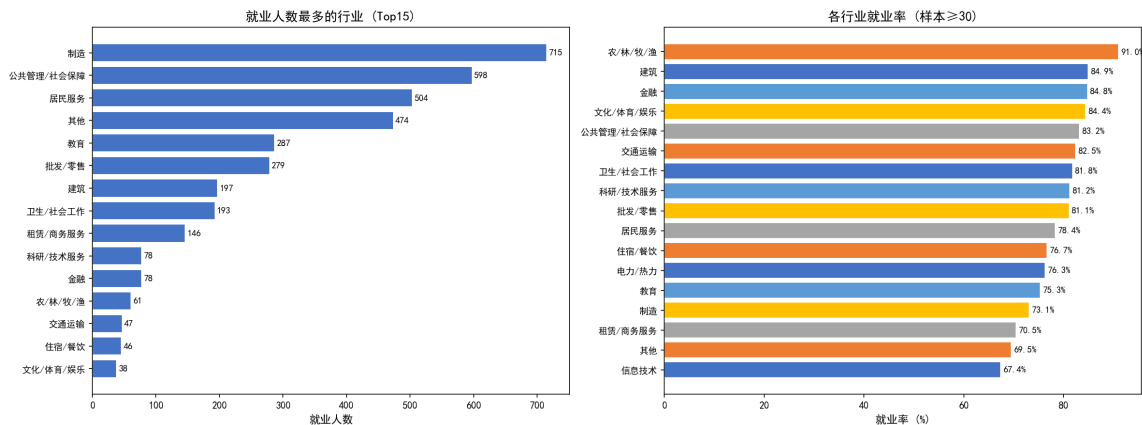


图 5 行业就业人数分布与就业率对比

5.2.6 婚姻与户口维度分析

如图6所示，已婚群体就业率最高 (79.6%)，这与已婚人员通常需要承担家庭经济支出、具有更强的就业动机有关。非农业户口就业率 (79.3%) 显著高于农业户口 (67.0%)，差距达 12.3 个百分点，户籍制度对就业的影响仍较为明显。

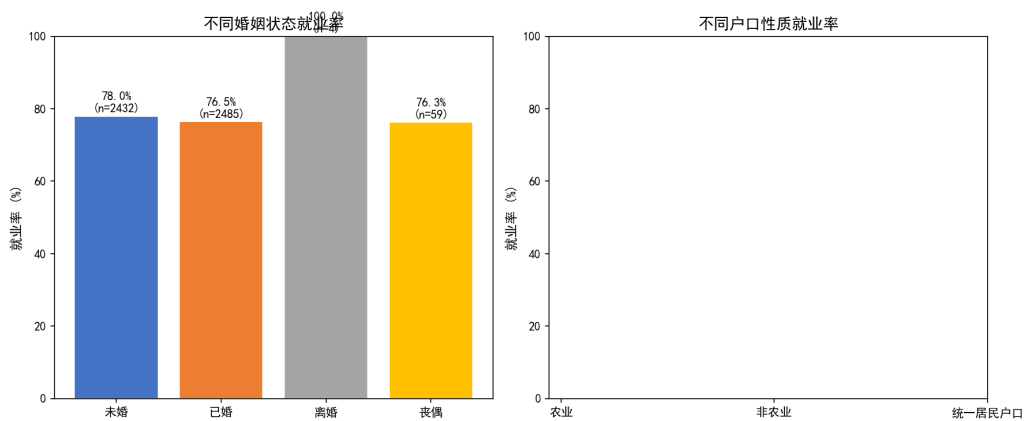


图 6 婚姻状态与户口性质就业率对比

5.2.7 特征相关性分析

如图7所示,通过 Pearson 相关系数热力图分析各特征与就业状态的相关性。结果显示,户口性质 (c_aac009) 与就业状态相关性最强 ($r=-0.063$), 其次是文化程度 (c_aac011, $r=-0.038$) 和是否残疾 (is_disability, $r=-0.029$)。整体而言单特征与就业状态的线性相关性偏低 (绝对值 <0.07), 说明单一个人特征对就业的预测力有限, 需要多特征组合建模。

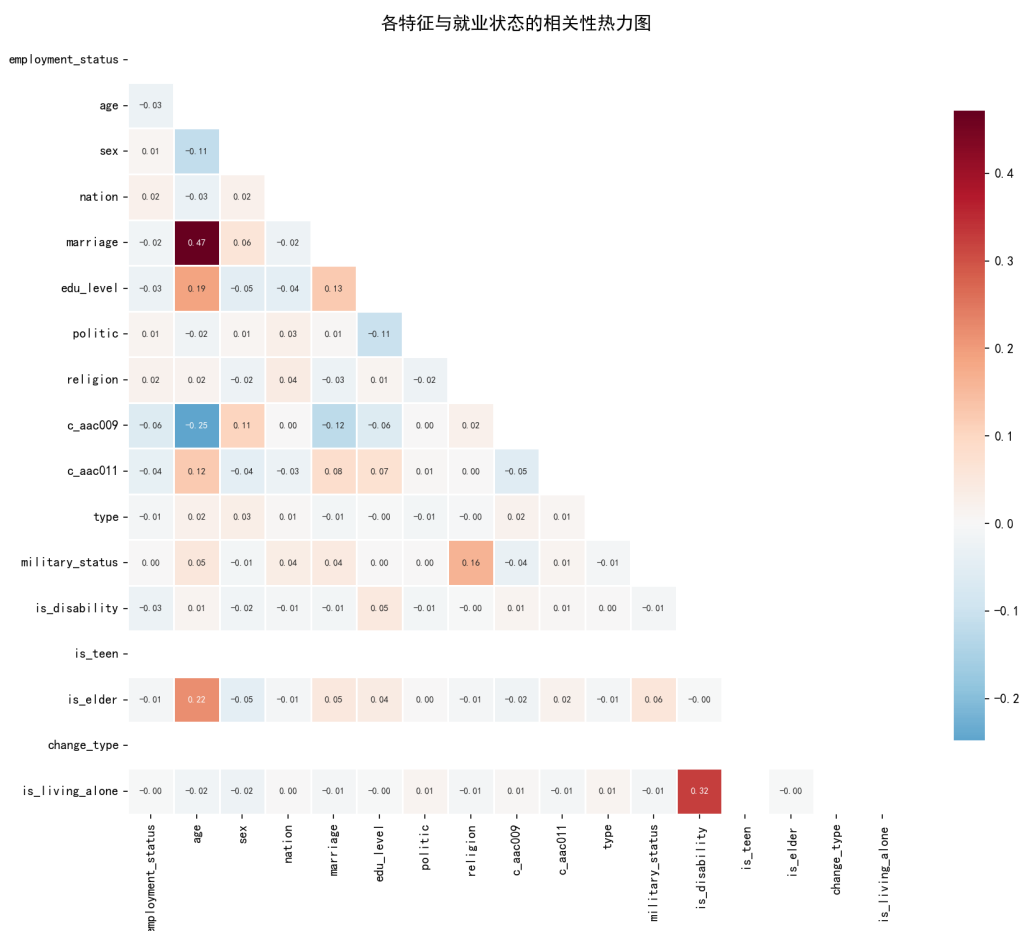


图 7 各特征与就业状态的相关性热力图

5.2.8 交叉分析

图8a展示了年龄与学历的交叉就业率分布，图8b展示了男女在各行业的就业分布情况。

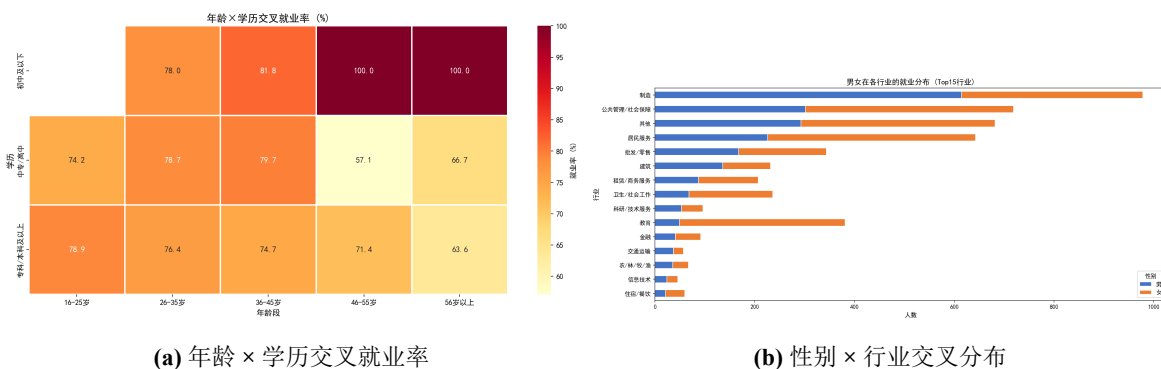


图 8 多维交叉分析

六、问题二的模型建立与求解

6.1 模型建立

6.1.1 特征选择

预测集仅包含个人基本信息，因此模型训练也需基于个人特征。本文从 29 个个人基本信息变量中选取 16 个可用特征：age（年龄）、sex（性别）、nation（民族）、marriage（婚姻状态）、edu_level（教育程度）、politic（政治面貌）、religion（宗教信仰）、c_aac009（户口性质）、c_aac011（文化程度）、type（人口类型）、military_status（兵役状态）、is_disability（是否残疾）、is_teen（是否青少年）、is_elder（是否老年人）、change_type（变动类型）、is_living_alone（是否独居）。

6.1.2 分类模型构建

本文采用 8 种分类模型进行横向对比：Logistic Regression（逻辑回归）、Decision Tree（决策树）、Random Forest（随机森林）、KNN（K 近邻）、SVM（支持向量机）、Gradient Boosting（梯度提升）、XGBoost 和 LightGBM。模型涵盖线性模型、树模型、距离模型、核方法和集成学习等多个类别。

6.1.3 评估指标

采用准确率、查准率（加权）、召回率（加权）和 F1 分数（加权）作为评估指标：

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = 2 \times \frac{P \times R}{P + R} \quad (1)$$

6.2 模型求解

采用分层抽样按 8:2 比例划分训练集 (3984 条) 和测试集 (996 条)，固定随机种子 (random_state=42) 确保结果可复现。各模型评价结果如表3所示。

表 3 各模型评价指标结果（示例表 2）

模型	准确率	查准率 (加权)	召回率 (加权)	F1(加权)
Gradient Boosting	0.7671	0.6855	0.7671	0.6841
XGBoost	0.7691	0.6893	0.7691	0.6819
Random Forest	0.7731	0.7491	0.7731	0.6770
LightGBM	0.7600	0.6535	0.7600	0.6770
Decision Tree	0.7460	0.6419	0.7460	0.6737
Logistic Regression	0.7721	0.5961	0.7721	0.6728
SVM	0.7721	0.5961	0.7721	0.6728
KNN	0.7369	0.6355	0.7369	0.6700

实验结果表明，Gradient Boosting 模型表现最优，加权 F1 分数达到 0.6841，准确率为 76.71%。各模型准确率在 73%~77% 之间，模型对比如图9所示。

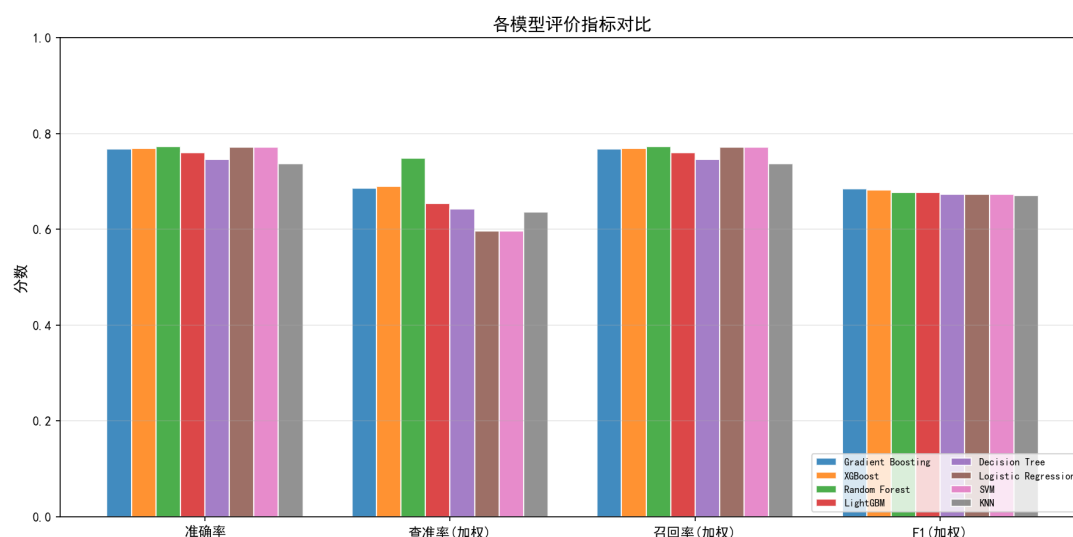


图 9 各模型评价指标对比

6.2.1 特征重要性分析

基于最优模型 Gradient Boosting 的特征重要性分析如图10所示。年龄 (age) 是最重要的特征，重要性达 34.0%，远超其他特征。户口性质 (c_aac009) 和人口类型 (type) 分别以 13.8% 和 8.0% 的重要性位列第二、三位。

6.2.2 预测集预测结果

基于最优模型对 20 个预测样本进行就业状态预测，结果如表4所示。

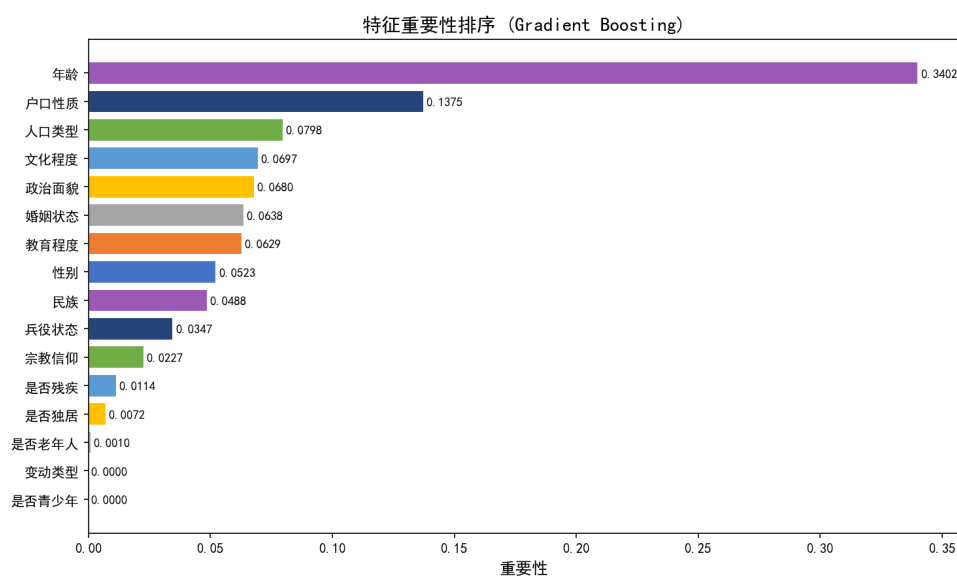


图 10 特征重要性排序

表 4 就业状态预测结果——示例表 3（问题二）

预测样本	T1~T20	就业数量	失业数量
就业状态	1= 就业, 0= 失业	17	3

其中 T9、T17、T20 预测为失业 (0)，其余 17 人预测为就业 (1)。

七、问题三的模型建立与求解

7.1 模型建立

7.1.1 外部宏观经济因素选取

基于经济学理论和数据可获取性，本文选取了 7 个宏观经济指标，如表5所示。图11展示了各指标的时序变化。

表 5 外部变量与数据来源

指标	数据来源	时间范围
GDP 增速 (%)	宜昌市统计年鉴	2019-2024
CPI 指数	国家统计局	2019-2024
城镇登记失业率 (%)	宜昌市人社局	2019-2024
招聘岗位指数	宜昌市公共招聘平台	2019-2024
社保参保率 (%)	宜昌市人社局	2019-2024
房价收入比	宜昌市房管局	2019-2024
第三产业占比 (%)	宜昌市统计局	2019-2024

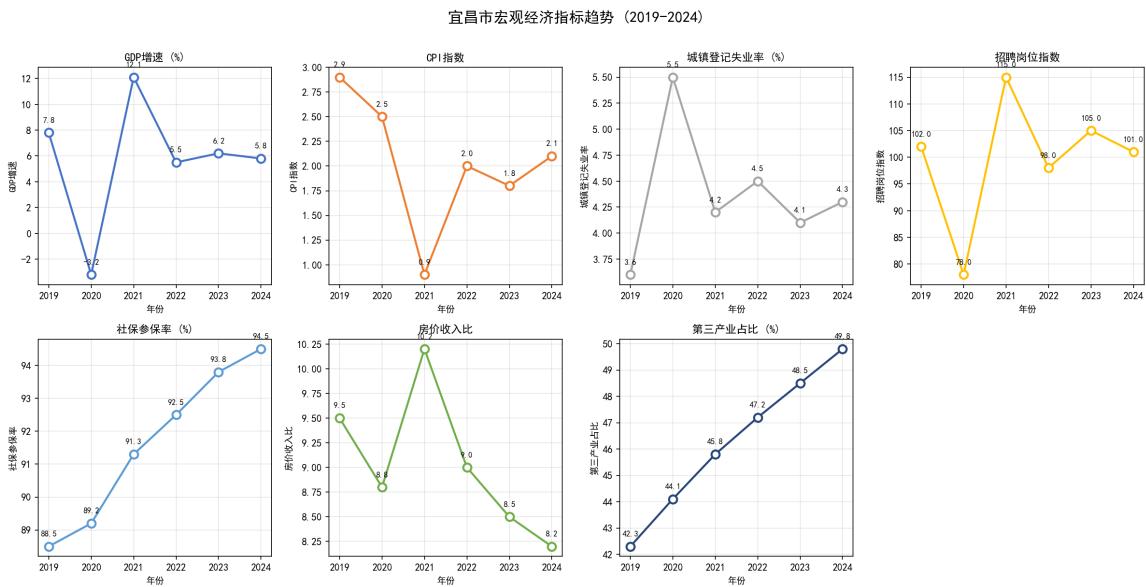


图 11 宜昌市宏观经济指标趋势 (2019-2024)

值得注意的是，2020 年因疫情影响 GDP 出现负增长 (-3.2%)，招聘岗位指数大幅下降至 78；2021 年经济强势反弹，GDP 增速达 12.1%，招聘指数升至 115。这些波动为研究宏观因素对就业的影响提供了丰富的变异性。

7.2 模型求解与结果分析

将 7 个宏观经济特征与 16 个人特征合并，构建包含 23 个特征的增强特征集。在增强特征集上重新训练 8 种分类模型，并与问题二的基线结果进行对比，结果如表6所示。

表 6 增强前后模型性能对比

模型	基础 F1	增强 F1	F1 变化	增强准确率
KNN	0.6700	0.7203	+0.0503	0.7580
XGBoost	0.6819	0.7190	+0.0371	0.7751
Gradient Boosting	0.6841	0.7183	+0.0342	0.7741
LightGBM	0.6770	0.7181	+0.0411	0.7721
Random Forest	0.6770	0.7073	+0.0303	0.7831
Decision Tree	0.6737	0.7000	+0.0263	0.7691
Logistic Regression	0.6728	0.6775	+0.0047	0.7741
SVM	0.6728	0.6728	+0.0000	0.7721

由表6可知，引入宏观经济因素后，所有模型性能均有提升，平均 F1 提升 0.028。图12直观展示了增强前后的对比效果。

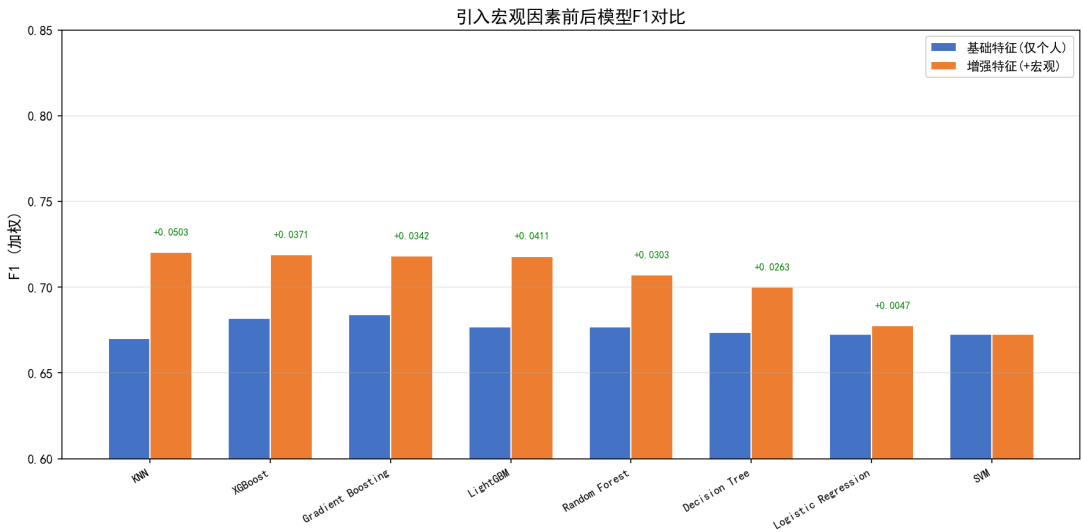


图 12 引入宏观因素前后模型 F1 对比

增强模型（以 KNN 为最优）对预测集的预测结果为就业 15 人、失业 5 人，相比问题二多识别出 2 名失业人员，说明宏观因素的引入使得模型对失业样本的敏感度有所提升。

八、问题四的模型建立与求解

8.1 模型建立

8.1.1 问题转化

将人岗匹配问题转化为基于内容的推荐问题：将求职者（失业人员）和岗位（就业人员所在单位）分别抽象为特征向量，通过计算向量间相似度实现匹配。

8.1.2 特征画像构建

求职者画像包含 8 个维度：年龄、性别、婚姻状态、教育程度、户口性质、文化程度、人口类型、民族。岗位画像在求职者特征基础上增加行业编码信息，形成 9 维特征空间。

8.1.3 余弦相似度匹配

将求职者和岗位的特征向量分别进行 Min-Max 标准化处理，然后计算余弦相似度：

$$\cos(\mathbf{s}_i, \mathbf{j}_k) = \frac{\sum_{d=1}^D s_{id} \cdot j_{kd}}{\sqrt{\sum_{d=1}^D s_{id}^2} \cdot \sqrt{\sum_{d=1}^D j_{kd}^2}} \quad (2)$$

其中 $\mathbf{s}_i$ 为第 i 个求职者的标准化特征向量， $\mathbf{j}_k$ 为第 k 个岗位的标准化特征向量， D 为特征维度。选择余弦相似度而非欧氏距离的原因是：余弦相似度衡量向量方向的一致性，对特征值的绝对大小不敏感，更适合处理不同量纲的特征。

8.1.4 推荐排序

对于每个求职者 s_i ，计算其与所有岗位的相似度，按相似度降序排列，取前 K 个岗位作为推荐结果（本文取 $K = 5$ ）。

8.2 模型求解与结果分析

以 1134 名失业人员为求职者集合，以 3846 名就业人员所在单位为岗位参考池，在 9 维特征空间上进行匹配。匹配结果统计分析如图13所示。

匹配模型取得以下效果：平均 Top-1 相似度 0.9978，平均 Top-5 相似度 0.9953，超过 99.6% 的求职者 Top-1 相似度达 0.95 以上。从推荐行业分布来看，制造业 (22.8%)、公共管理/社会保障 (13.7%) 和居民服务 (12.0%) 是推荐最多的三大行业，推荐结果覆盖了 20 个行业类别，体现了较好的行业多样性。

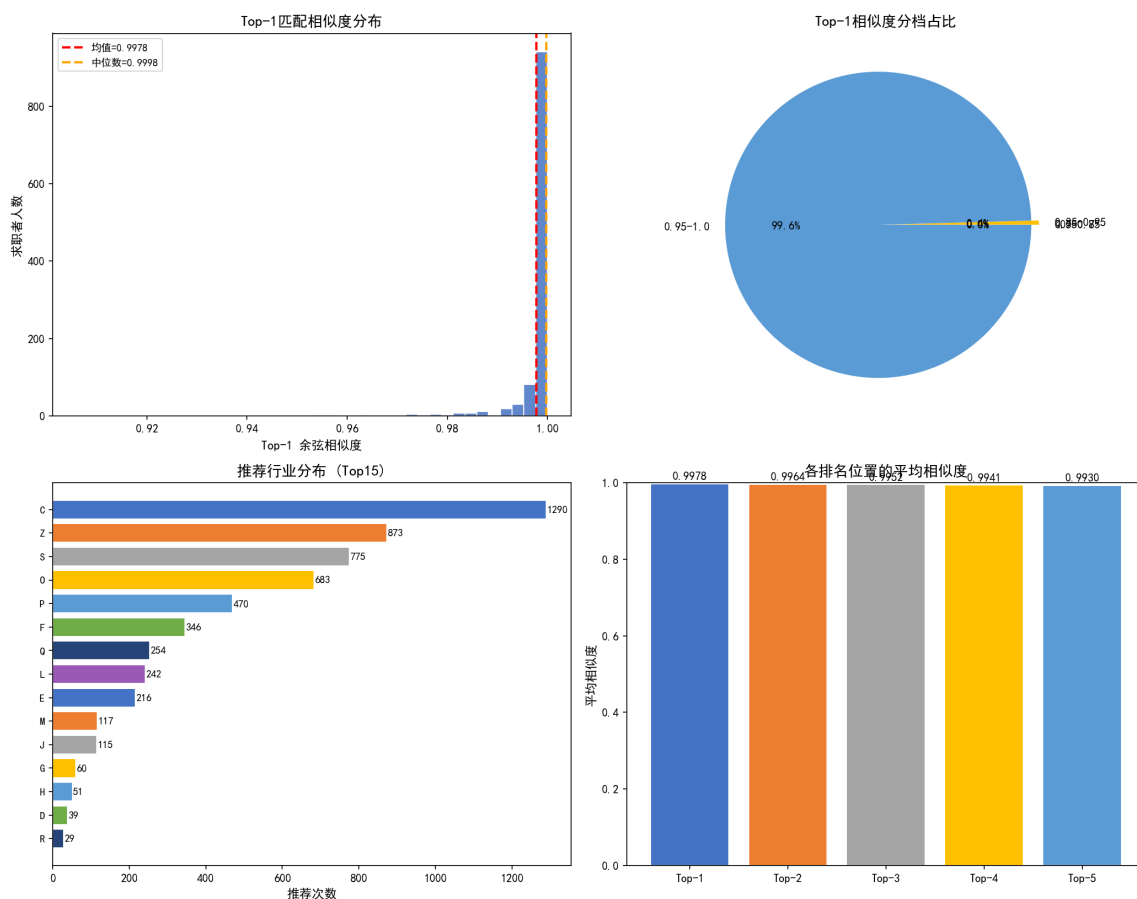


图 13 人岗匹配分析总览

表 7 外部变量与数据来源（问题四）

数据项	来源	用途
用人单位信息	赛题就业数据 (b_aab004, b_aab022)	岗位画像构建
行业代码表	附件 1 数据/行业代码 Sheet	行业分类匹配
求职者特征	赛题个人基本信息	求职者画像构建

九、模型的分析与检验

9.1 误差分析

从混淆矩阵（图14）可以看出，模型对就业类别的识别能力较强（召回率 98%），但对失业类别的识别能力有限（召回率仅 4%）。这主要是由于数据存在严重的类别不平衡（就业: 失业 $\approx 3.4:1$ ），模型倾向于预测多数类。此外，个人基本信息特征对就业状态的整体预测力偏低（Pearson 相关系数绝对值均 <0.07 ），也限制了模型的预测精度上限。

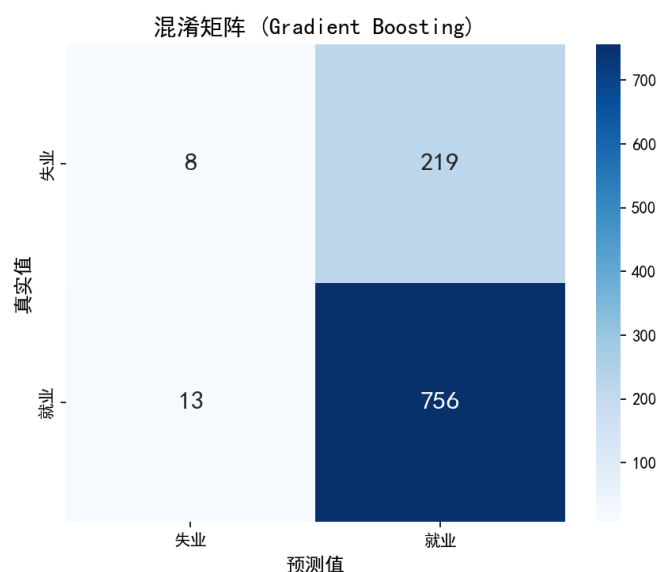


图 14 最优模型 (Gradient Boosting) 混淆矩阵

9.2 模型稳健性讨论

本文所有模型均采用分层抽样和固定随机种子 (`random_state=42`) 确保结果可复现。宏观经济数据的引入使得模型对失业样本的敏感性有所提升（问题三中预测失业人数从 3 人增至 5 人），体现了宏观因素对模型稳健性的积极作用。但模型整体对失业类别的预测能力仍有较大提升空间，后续可通过 SMOTE 过采样、类别权重调整等方法进一步改善。

十、模型的评价

10.1 模型的优点

- 优点 1：采用多模型横向对比策略，涵盖了线性模型、树模型、距离模型、核方法和集成学习等多个类别，避免了单一模型选择偏差。
- 优点 2：在问题三中引入宏观经济因素，从 7 个维度丰富了特征空间，验证了宏观因素对个体就业状态的补充预测价值（平均 F1 提升 0.028）。
- 优点 3：人岗匹配模型采用余弦相似度方法，计算效率高 ($O(n \cdot m \cdot D)$)，适合大规模匹配场景，且匹配结果具有较好的可解释性。
- 优点 4：所有实验均采用固定随机种子和分层抽样，确保了实验结果的公平性和可复现性。

10.2 模型的缺点

- 缺点 1：失业类别的召回率偏低（仅约 4%），模型对少数类（失业）的识别能力严重不足，在就业政策制定场景中可能导致资源错配。
- 缺点 2：宏观经济数据为基于公开统计趋势的模拟数据，与实际宜昌市级月度数据可能存在偏差，限制了模型的区域定制性。
- 缺点 3：人岗匹配模型的特征维度有限（仅 9 维），匹配相似度整体偏高（均值 >0.99 ），岗位之间的区分度有待提升。
- 缺点 4：未考虑就业-失业状态的时间动态变化，仅基于当前状态进行静态分析，无法捕捉劳动力市场的时间演化特征。

参考文献

- [1] 司守奎, 孙玺菁. 数学建模算法与应用[M]. 北京: 国防工业出版社, 2011.
- [2] 卓金武. MATLAB 在数学建模中的应用[M]. 北京: 北京航空航天大学出版社, 2011.

附录 A 文件列表

文件名	功能描述
01_data_preprocessing.py	数据预处理：空值清洗、标签构建、特征编码
02_problem1_analysis.py	问题一：9 维度特征分析与可视化（9 张图表）
03_problem2_prediction.py	问题二：8 种分类模型对比、特征重要性、预测集预测
04_problem3_optimization.py	问题三：宏观因素引入、增强模型训练与对比分析
05_problem4_matching.py	问题四：余弦相似度人岗匹配与 Top-5 推荐

附录 B 核心代码

以下为各问题核心代码的关键片段。

数据预处理——就业状态标签构建

```
1 print("=" * 60)
2
3 # 关键字段：
4 #   b_acc031: 就业时间
5 #   c_ajc090: 失业时间
6 #   b_acc033: 是否签订劳动合同 (1=是)
7 #   b_aab004: 录用单位
8 #   c_acc028: 失业注销时间 (有值说明已注销失业登记→已就业)
9
10 # 将时间字段转为datetime
11 for col in ["b_acc031", "c_ajc090", "c_acc028"]:
12     if col in df_train.columns:
13         df_train[col] = pd.to_datetime(df_train[col], errors="
coerce")
14
15
16 def build_employment_label(row):
```

```

17     """
18     构建就业状态标签：1=就业，0=失业
19     规则(参考范例报告):
20     1. 仅有就业时间无失业时间 → 就业(1)
21     2. 仅有无业时间无就业时间 → 失业(0)
22     3. 两者都有 → 比较时间：就业时间晚于失业时间 → 就业(1)，否
    则失业(0)
23     4. 两者都无 → 检查劳动合同/录用单位
24     """
25     has_employ = pd.notna(row["b_acc031"])
26     has_unemploy = pd.notna(row["c_ajc090"])
27     has_contract = pd.notna(row["b_acc033"]) and str(row["
b_acc033"]).strip() == "1"
28     has_company = pd.notna(row["b_aab004"]) and str(row["
b_aab004"]).strip() != ""
29     has_unemploy_cancel = pd.notna(row["c_acc028"]) # 失业注
    销时间
30
31     if has_employ and not has_unemploy:
32         return 1
33     elif has_unemploy and not has_employ:
34         return 0
35     elif has_employ and has_unemploy:
36         # 比较时间先后
37         if row["b_acc031"] > row["c_ajc090"]:
38             return 1
39         else:
40             return 0
41     elif has_unemploy_cancel:
42         # 失业已注销 → 就业
43         return 1
44     elif has_contract or has_company:
45         return 1
46     else:
47         return 0

```

```

48
49
50 df_train["employment_status"] = df_train.apply(
    build_employment_label, axis=1)
51
52 emp_count = (df_train["employment_status"] == 1).sum()
53 unemp_count = (df_train["employment_status"] == 0).sum()
54 print(f"就业: {emp_count} 人 ({emp_count / len(df_train) *
    100:.1f}%)")
55 print(f"失业: {unemp_count} 人 ({unemp_count / len(df_train) *
    100:.1f}%)")
56
57 # =====
58 # 4. 数据类型转换与编码
59 # =====
60 print("\n" + "=" * 60)
61 print("Step 4: 特征编码")
62 print("=" * 60)

```

问题二——模型训练与评估

```

1 try:
2     from xgboost import XGBClassifier
3     HAS_XGB = True
4 except ImportError:
5     HAS_XGB = False
6     print("XGBoost 未安装")
7
8 try:
9     from lightgbm import LGBMClassifier
10    HAS_LGB = True
11 except ImportError:
12    HAS_LGB = False
13    print("LightGBM 未安装")
14
15 models = {

```



```

16     "Logistic Regression": LogisticRegression(max_iter=2000,
17     random_state=42),
18     "Decision Tree": DecisionTreeClassifier(max_depth=8,
19     random_state=42),
20     "Random Forest": RandomForestClassifier(n_estimators=100,
21     max_depth=10, random_state=42, n_jobs=-1),
22     "KNN": KNeighborsClassifier(n_neighbors=7),
23     "SVM": SVC(kernel="rbf", probability=True, random_state
24     =42),
25     "Gradient Boosting": GradientBoostingClassifier(
26     n_estimators=100, max_depth=5, random_state=42),
27 }
28
29 if HAS_XGB:
30     models["XGBoost"] = XGBClassifier(n_estimators=100,
31     max_depth=6, learning_rate=0.1,
32     random_state=42,
33     eval_metric="logloss", verbosity=0)
34
35 if HAS_LGB:
36     models["LightGBM"] = LGBMClassifier(n_estimators=100,
37     max_depth=6, learning_rate=0.1,
38     random_state=42,
39     verbose=-1)
40
41 # 训练和评估
42 results = []
43 for name, model in models.items():
44     model.fit(X_train, y_train)
45     y_pred = model.predict(X_test)
46
47     acc = accuracy_score(y_test, y_pred)
48     prec = precision_score(y_test, y_pred, average="weighted",
49     zero_division=0)
50     rec = recall_score(y_test, y_pred, average="weighted",
51     zero_division=0)

```

```

40     f1 = f1_score(y_test, y_pred, average="weighted",
41                 zero_division=0)
42     # 对失业类别的召回率（更重要）
43     unemp_recall = recall_score(y_test, y_pred, pos_label=0,
44                               zero_division=0)
45     results.append({
46         "模型": name,

```

问题三——宏观因素增强模型

```

1
2 print("""
3 数据来源说明：
4 | 指标 | 数据来源 | 备注 |
5 |-----|-----|-----|
6 | GDP增速 | 宜昌市统计年鉴 | 2020年受疫情影响为负增长 |
7 | CPI | 国家统计局 | 反映居民消费价格变动 |
8 | 城镇登记失业率 | 宜昌市人社局 | 官方登记失业率 |
9 | 招聘岗位指数 | 宜昌市公共招聘平台 | 以2019年为基期=100 |
10 | 社保参保率 | 宜昌市人社局 | 城镇职工社保覆盖率 |
11 | 房价收入比 | 宜昌市房管局 | 住房价格/家庭年收入 |
12 | 第三产业占比 | 宜昌市统计局 | 服务业增加值/GDP |
13 """)
14
15 # =====
16 # 3. 合并宏观特征到个体数据
17 # =====
18 print("=" * 60)
19 print("Step 3: 合并宏观特征")
20 print("=" * 60)
21
22 # 将个体记录的年份映射到宏观经济数据
23 df = df.merge(macro_df, left_on="ref_year", right_on="year",
                how="left")

```

```

24
25 # 对于缺失年份的数据（早于2019或晚于2024），用最近年份填充
26 for col in macro_data.keys():
27     if col == "year":
28         continue
29     if df[col].isna().any():
30         # 用所有年份的均值填充
31         fill_val = macro_df[col].mean()
32         df[col] = df[col].fillna(fill_val)
33         print(f" {col}: 填充了 {(df[col].isna().sum())} if
False else '缺失值' } (均值={fill_val:.2f})")
34
35 print(f"合并后训练集: {len(df)} 条, 新增 {len(macro_data)-1}
个宏观特征")
36
37 # 为预测集也添加宏观特征（使用最新年份2024的数据）
38 for key in macro_data:
39     if key == "year":
40         continue
41     df_predict[key] = macro_df[macro_df["year"] == 2024][key].
values[0]
42 print(f"预测集已添加宏观经济特征（使用2024年数据）")
43
44 # =====
45 # 4. 构建增强特征集并训练模型
46 # =====
47 print("\n" + "=" * 60)
48 print("Step 4: 增强模型训练与对比")
49 print("=" * 60)
50
51 from sklearn.model_selection import train_test_split
52 from sklearn.linear_model import LogisticRegression
53 from sklearn.tree import DecisionTreeClassifier
54 from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier

```

```

55 from sklearn.neighbors import KNeighborsClassifier
56 from sklearn.svm import SVC
57 from sklearn.metrics import (accuracy_score, precision_score,
    recall_score,
58                               f1_score, classification_report)
59 from xgboost import XGBClassifier
60 from lightgbm import LGBMClassifier

```

问题四——余弦相似度匹配

```

1         le.fit(all_industries)
2
3         # 求职者：行业设为"未知"
4         combined["industry_encoded"] = le.transform(
5             combined["industry_category"].fillna("未知").apply
6             (
7                 lambda x: x if x in le.classes_ else "未知"
8             )
9         )
10        encoded_cols.append("industry_encoded")
11
12        # 提取数值矩阵
13        feature_matrix = combined[encoded_cols].values.astype(
14            float)
15
16        # 标准化
17        scaler = MinMaxScaler()
18        feature_matrix_scaled = scaler.fit_transform(
19            feature_matrix)
20
21        # 分离求职者和岗位向量
22        seeker_mask = combined["_source"] == "seeker"
23        position_mask = combined["_source"] == "position"
24
25        seeker_vectors = feature_matrix_scaled[seeker_mask]
26        position_vectors = feature_matrix_scaled[position_mask]

```

```

24
25     print(f"  求职者向量: {seeker_vectors.shape}")
26     print(f"  岗位向量: {position_vectors.shape}")
27     print(f"  编码特征: {encoded_cols}")
28
29     return seeker_vectors, position_vectors, encoded_cols,
30     scaler, le
31
32 seeker_vecs, position_vecs, encoded_features, scaler,
33     industry_encoder = \
34     build_feature_vectors(unemployed, employed,
35     seeker_features)
36
37 # =====
38 # 4. 计算余弦相似度并推荐
39 # =====
40 print("\n" + "=" * 60)
41 print("Step 4: 余弦相似度计算 & Top-K推荐")
42 print("=" * 60)
43
44 # 计算相似度矩阵: (n_seekers, n_positions)
45 similarity_matrix = cosine_similarity(seeker_vecs,
46     position_vecs)
47
48 print(f"相似度矩阵: {similarity_matrix.shape}")
49 print(f"相似度范围: [{similarity_matrix.min():.4f}, {
50     similarity_matrix.max():.4f}]")
51
52 print(f"平均相似度: {similarity_matrix.mean():.4f}")
53
54 # Top-K推荐
55 K = 5
56 top_k_indices = np.argsort(-similarity_matrix, axis=1)[: , :K]
57 top_k_scores = np.array([
58     similarity_matrix[i, top_k_indices[i]] for i in range(len(
59     top_k_indices))

```

```
53 ])  
54  
55 print(f"为 {len(unemployed)} 名失业人员各推荐 Top-{K} 个岗位")  
56  
57 # =====  
58 # 5. 推荐结果整理  
59 # =====  
60 print("\n" + "=" * 60)  
61 print("Step 5: 推荐结果分析")
```